

SPRINT 12 · TOOL-USING AGENTS · SESIÓN 1

El agente actúa

El modelo pide, Python ejecuta. Implementamos la allowlist de la Live Review mientras repasamos la teoría.

HOY, EN 120 MINUTOS

Del agente que habla al agente que hace

0-10	Apertura	De S11 a S12 · los dos bucles
10-38	Pinceladas 1 y 2	Qué es una tool · function calling y su ciclo
38-55	Live Review · Fase 0	Entorno, contrato y --check en rojo
55-65	Descanso	
65-88	Las tools que vienen dadas	hora, mini-RAG y API + fallback, probadas sin el modelo
88-115	Live Review · Fase 1	Escribimos la allowlist y la validación
115-120	Cierre	Reto para la próxima sesión

DE DÓNDE VENIMOS

Dos bucles que no hay que mezclar

Bucle de conversación · S11

El **usuario** da el siguiente paso escribiendo otro mensaje.
Lo limita **max_turns**.

Bucle de tools · S12

El **agente** pide herramientas **dentro** de un mismo turno
hasta tener respuesta. Lo limita **max_steps**.

Pregunta del sprint: **¿cómo deajo que el agente use herramientas y vuelva con un resultado, sin perder el control?**

Una función real que el modelo puede pedir

Nombre	Identificador estable	hora_actual
Descripción	Cuándo usarla: la lee el modelo	“Devuelve la fecha y hora local actuales”
Parámetros	Schema JSON: tipos y obligatorios	Ninguno
Implementación	Código Python que tú controlas	<code>datetime.now().isoformat()</code>

Si no hay una función real detrás, no hay tool use: **hay teatro**.

Cada una evita que el modelo finja algo

hora_actual

El modelo no tiene reloj. Sin tool, se inventa la hora con total seguridad.

RAG_buscar_en_guia

Mini-RAG léxico sobre la guía de calidad del aire. Lo que diga sale del corpus, no de su memoria.

API_consultar_calidad_aire

Mediciones reales del open data de Madrid y, si falla, un JSON local. Nunca inventadas.

El ciclo: pedir, ejecutar, devolver



El modelo

Devuelve un **function call**: nombre y argumentos. No ejecuta nada.

Tu código

Decide si lo ejecuta, lo ejecuta y devuelve el resultado.

Puede repetirse

Varias vueltas antes del texto final. Por eso existe `max_steps`.

Sin encadenar, el modelo olvida lo que pidió

user	Instrucciones y mensaje del usuario
model	El function call (o el texto final)
tool	El resultado que ha calculado Python

Automático (AFC)

El SDK ejecuta las tools por ti. Cómodo, pero opaco.

Manual, el de la práctica

run_tool_loop con AFC desactivado: vemos allowlist, traza y max_steps.

Agente de calidad del aire, ahora con tools

Carpeta: Practica_live_review/01_agente_tools_calidad_aire · mismo dominio que el Sprint 11, con herramientas

Lo que viene dado

state.py, las tres tools, casi todo
llm.py, main.py, app.py y
verificar.py

Lo que implementas

La allowlist en src/tools/__init__.py,
el hueco de run_tool_loop y
src/agent.py entero

Cuatro fases

Entorno, allowlist, loop y
orquestración, demostrar en CLI y
Streamlit.

Arrancamos con el check en rojo

```
cd Practica_live_review/01_agente_tools_calidad_aire
python -m venv .venv
source .venv/bin/activate
# Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt
cp .env.example .env # pega tu GEMINI_API_KEY
python main.py --check
```

Seis FAIL y un OK:

```
[FAIL] TOOL_REGISTRY
[FAIL] validar_args
[FAIL] ejecutar_tool
[OK] API (dada)
[FAIL] procesar_turno
[FAIL] run_demo
[FAIL] run_tool_loop
```

FASE 0 · UN TURNO POR DENTRO

Dos fases y una decisión en Python



La tool recupera; el modelo genera

Lo que hace

Carga la guía de calidad del aire, la parte por secciones ##, puntúa por palabras en común y devuelve los dos mejores trozos.

Lo que no hace

Ni embeddings, ni base vectorial, ni redactar la respuesta. Si mañana metes Chroma, el contrato no cambia.

Su límite, en directo

Es recuperación léxica: si la palabra no aparece, no encuentra nada, y si aparece por casualidad, acierta por casualidad.

El mundo exterior falla; la práctica no



La fila cruda trae H01...H24 y V01...V24. La tool se queda con la **última hora validada** y devuelve una línea por estación: contaminante, valor y fecha. **Nunca el JSON entero.**

DEMO EN DIRECTO

Las tools funcionan sin el modelo

```
python
from src.tools.hora_actual import hora_actual
from src.tools.rag_guia import RAG_buscar_en_guia
from src.tools.api_calidad_aire import API_consultar_calidad_aire

hora_actual()
RAG_buscar_en_guia("NO2 valor alto")
API_consultar_calidad_aire(limite=2)
```

Las importamos directas porque ejecutar_tool todavía es un TODO. Si una tool no se puede probar sin el modelo, está mal diseñada.

La allowlist y la validación

```
TOOL_REGISTRY = {
    "hora_actual": hora_actual,
    "RAG_buscar_en_guia": RAG_buscar_en_guia,
    "API_consultar_calidad_aire": API_consultar_calidad_aire,
}

def validar_args(nombre, args=None):
    args = args or {}
    if nombre == "hora_actual": return {}
    if nombre == "RAG_buscar_en_guia":
        return {"consulta": str(args.get("consulta") or "").strip()}
    if nombre == "API_consultar_calidad_aire":
        try: limite = int(args.get("limite", 5))
        except (TypeError, ValueError): limite = 5
        return {"limite": max(1, min(limite, 10))}
    return args
```

FASE 1 · UNA SOLA PUERTA

ejecutar_tool: lo único que llama a una tool

```
def ejecutar_tool(nombre, args=None):
    if nombre not in TOOL_REGISTRY:
        return f"Error: tool '{nombre}' no permitida."
    limpios = validar_args(nombre, args)
    if nombre == "RAG_buscar_en_guia" and not limpios.get("consulta"):
        return "Error: la consulta de RAG no puede estar vacía."
    try:
        return TOOL_REGISTRY[nombre](**limpios)
    except Exception as e:
        return f"Error al ejecutar {nombre}: {e}"
```

Comprobamos:

```
ejecutar_tool("marcar_done")
validar_args("API_...", {"limite": 99})
python main.py --check
```

Tres OK nuevos; siguen en rojo el loop y el agente.

CIERRE DE LA SESIÓN 1

Tres ideas y un reto

1

Una tool es una función real con contrato: nombre, descripción y parámetros.

2

El modelo pide; Python ejecuta y le devuelve el resultado en el historial.

3

La allowlist es una puerta única: si no está registrada, no se ejecuta.

Reto: intentad el TODO de `run_tool_loop` en `llm.py`. Son seis líneas y están en las pistas plegadas del README.

SPRINT 12 · TOOL-USING AGENTS · SESIÓN 2

El loop, los topes y la traza

Cerramos el agente: el bucle de tools, procesar_turno, run_demo, los cortes, Streamlit y LangGraph.

HOY, EN 120 MINUTOS

Terminar el agente y demostrarlo

0-10	Repaso relámpago	Cinco preguntas en frío
10-30	Fase 2a	El hueco de run_tool_loop y el status blocked
30-55	Fase 2b	procesar_turno y run_demo en directo
55-65	Descanso	
65-95	Fase 3	Demo, traza de tools, cortes y fallback
95-108	Fase 4 y LangGraph	Streamlit y los dos notebooks
108-120	Cierre	Criterio de cierre y puente al Sprint 13

REPASO RELÁMPAGO

Contestad en el chat, sin mirar apuntes

- 1 ¿El modelo ejecuta las tools?
- 2 ¿Qué tres roles tiene el historial del bucle de tools?
- 3 ¿Qué hace validar_args con limite = 99?
- 4 ¿Qué devuelve ejecutar_tool si la tool no está registrada?
- 5 ¿Por qué la API devuelve un resumen y no el JSON?

El modelo pide. Python ejecuta.

El LLM elige qué tool quiere y con qué argumentos. Si se ejecuta, y con qué valores, lo decide tu código: allowlist y validación.

El hueco de run_tool_loop

```
for llamada in llamadas:
    nombre = llamada.name
    args = dict(llamada.args or {})

    if nombre not in TOOL_REGISTRY:
        resultado = f"Error: tool '{nombre}' no está permitida."
        status = "blocked"
    else:
        resultado = ejecutar_tool(nombre, args)
        status = "error" if resultado.startswith("Error") else "ok"

    traza.append({"step": step, "tool": nombre, "args": args,
                  "status": status, "preview": _preview(resultado)})
```

Dos barreras: esta, que marca **blocked** en la traza, y la de ejecutar_tool, que es la última.

Quitamos una tool del registro

```
# src/tools/__init__.py
TOOL_REGISTRY = {
    # "hora_actual": hora_actual,
    "RAG_buscar_en_guia": RAG_buscar_en_guia,
    "API_consultar_calidad_aire": API_...,
}
```

El modelo la sigue viendo en el menú de _declaraciones, la pide... y en la traza sale **status: blocked**.

El menú no es la cocina.

procesar_turno: dos fases y una traza

```
n = len(estado.get("traza") or []) + 1
try:
    texto_tools, traza_tools = run_tool_loop(mensaje, max_steps=max_steps)
    cambios = sintetizar_estado(estado, mensaje, texto_tools)
    estado = actualizar_estado(estado, cambios)
    if not estado.get("respuesta"):
        estado = {**estado, "respuesta": texto_tools}
    estado = calcular_done(estado)
    estado.setdefault("traza", []).append(
        {"turno": n, "mensaje": mensaje, "status": "ok",
         "done": estado.get("done"), "tools": traza_tools})
except Exception as e:
    # error + respuesta amigable + traza status "error", tools=[]
```

run_demo: el mismo bucle del Sprint 11

```
msgs = list(mensajes) if mensajes is not None else list(config.DEMO_MENSAJES)
limit = max_turns if max_turns is not None else config.MAX_TURNS_DEFAULT
estado = crear_estado(msgs[0] if msgs else "")

for mensaje in msgs[:limit]:
    if estado.get("done") or estado.get("error"):
        break
    estado = procesar_turno(estado, mensaje, max_steps=max_steps)

# si cortó por tope y no hay done ni error: aviso en la respuesta
return estado
```

Ojo al tercer argumento: **max_steps viaja hasta procesar_turno**. Si se te olvida, el slider de Streamlit no hará nada.

FASE 3 · DOS TOPES DISTINTOS

max_steps frente a max_turns

python main.py --check	Todo [OK], incluido el mensaje vacío
python main.py	Tools en traza; preferencias, plan y a menudo done
python main.py --max-turns 2	Corte por turnos: se queda a medias del guion
python main.py --max-turns 1 --max-steps 1	Corte del bucle de tools dentro de un mensaje

max_turns cuenta **mensajes del usuario**; max_steps cuenta **vuelatas modelo ↔ tools** dentro de un mensaje.

Lo que te van a pedir en la revisión

```
{
  "turno": 3,
  "status": "ok", "done": false,
  "tools": [
    {
      "step": 1,
      "tool": "RAG_buscar_en_guia",
      "args": {"consulta": "NO2 interpretar"},
      "status": "ok",
      "preview": "### Contaminantes y códigos...",
    },
    {
      "step": 1,
      "tool": "API_consultar_calidad_aire", ...
    }
  ]
}
```

step: vuelta del bucle de tools.

args: lo que pidió el modelo.

status: ok, error o blocked.

preview: recorte del resultado.

Dos tools en el mismo step = el modelo las pidió en paralelo.

Rompemos el mundo exterior a propósito

API caída

En config.py, cambia MADRID_AIRE_URL por una URL que no exista. En el preview:

Fuente: fallback.

API key mala

Clave inventada en .env: el turno acaba con **error** y traza con status error, sin traceback.

Sin Wi-Fi, no

Quitar la red también tumba a Gemini y el turno falla antes de llegar a la tool.

La app viene dada: es un cliente

Dos sliders

max_turns frena los mensajes;
max_steps frena las vueltas de
tools por mensaje.

Expander

Estado y traza de tools: aquí se
enseña la Live Review.

Dos memorias

messages pinta el chat;
agent_state dirige la tarea. La traza
de tools no es el historial.

Si mañana cambias Streamlit por una API, **procesar_turno no se toca**.

Mismo dibujo, otros nombres



Lineal y con bifurcación

```
grafo = StateGraph(EstadoAgente)
grafo.add_node("normalizar", nodo_normalizar)
grafo.add_node("proponer", nodo_proponer)
grafo.add_edge(START, "normalizar")
grafo.add_edge("normalizar", "proponer")
grafo.add_edge("proponer", END)
app = grafo.compile()
```

```
grafo.add_conditional_edges(
    "clasificar", router)

def router(estado):
    return estado["intencion"]
# preferencias | eventos
# horario | general
```

CRITERIO DE CIERRE

Si lo explicas en voz alta, el sprint está cerrado

- 1 Qué hace `procesar_turno(estado, mensaje, max_steps)` y qué devuelve.
- 2 Quién pide la tool y quién la ejecuta.
- 3 Para qué sirven la `allowlist` y el `status blocked`.
- 4 Diferencia entre `max_steps` y `max_turns`.
- 5 Por qué `done` lo calcula Python y no es una tool.
- 6 Qué miras en `traza[].tools`: `step`, `tool`, `args`, `status`, `preview`.
- 7 Qué vive en `messages` y qué en `agent_state`.

QUÉ PROBAR ANTES DE ENTREGAR

La tabla del README, resumida

<code>python main.py --check</code>	Todo en [OK]
Demo completa	Hay tools en la traza y plan con preferencias
<code>--max-turns 2</code>	Para sin terminar el guion, con aviso
<code>--max-steps 1</code>	Tools muy limitadas en el turno
Mensaje vacío	Respuesta amable, sin LLM y sin traza nueva
Fallback	Fuente: fallback en el preview de la API

Ahora, autonomía con límites.

Migraremos este agente a LangGraph como orquestador, con las mismas tools, y añadiremos human in the loop: pedir permiso antes de actuar.